

FDA Medical Device Classification System

Final Report

Prepared by: Xinyan Xu

Baynovation

June 22, 2026

Table of Contents

1. Project Overview	4
2. Method	5
1. Overall Architecture: Hybrid Two-Step Approach	5
2. Detailed Description of Each Cell	5
Cell 1 — Import Libraries	5
Cell 2 — API Connection and Path Configuration	5
Cell 3 — Build the CFR Regulatory Index	6
Cell 4 — Define Embedding Utility Functions	6
Cell 5 — Generate/Load the CFR Vector Cache	6
Cell 6 — Define the Device File Reading Function	7
Cell 7 — Sort the 100 Files	7
Cell 8 — Query Rewriting + Embedding Retrieval	7
Cell 9 — Install pydantic	7
Cell 10 — Define the GPT Classification Function (Two-Step Method + Consistency Check)	7
Cell 11 — Merge the Two Steps into a Single Function	9
Cell 12 — Quick Single-Record Test	9
Cell 13 — Batch Evaluation (All 100 Files)	9
Cell 14 — Failure Case Analysis	10
Cell 15 — Export the Excel Results Table	10
3. Output Files	10
3. Results	11
1. Overall Accuracy	11
2. Funnel Breakdown	11
3. Internal Consistency Check	12
4. Failure Attribution	12
5. Confidence Calibration	14
6. Representative Cases	14
4. Observations	15
1.Errors are concentrated in a small number of regulations rather than randomly	

distributed.	15
2.Accuracy varies considerably across CFR Parts, but sample sizes are too small to draw conclusions.	15
3.Confidence is generally high and shows no reliable relationship with correctness or reasoning length.	15
4.The two-step design performed as expected in this run.	15
5. Future Directions	16
1. Retrieval Stage (addressing the 17 retrieval failures)	16
2. Classification Stage (addressing the 12 cases where the regulation was correct but the code was wrong)	16

1. Project Overview

Core task: given a device's free-text description, automatically determine which regulation number under FDA 21 CFR Title 21, Subchapter H the device should be classified under, along with the corresponding product code.

Data sources: (1) a CFR JSON file organized into a four-level hierarchy — Part → Subpart → Entry → Product Code — covering 2,101 regulations and 5,955 product codes, which serves as the knowledge base for the classification system; and (2) a 100-record test dataset, where each record corresponds to a real 510(k) submission summary containing a device description along with its correct regulation number and product code, used as ground truth for evaluation.

2. Method

Terminology note: every reference to "Cell N" in this document refers to the number labeled in the comment at the top of the code block (i.e., written as # Cell N: ...), not the physical cell order in the Jupyter notebook.

1. Overall Architecture: Hybrid Two-Step Approach

The system uses a hybrid approach combining "embedding vector retrieval + GPT two-step classification."

Step	Method	Description
Step 1	Embedding vector retrieval	Converts the device description into a vector, computes cosine similarity against the vectors of all 5,955 product codes in the regulatory database, and selects the top 40 candidates with the highest similarity.
Step 2	GPT two-step classification	Groups the 40 candidates by their parent regulation number and sends them, together with the device description, to GPT-4.1. GPT first selects the best-matching regulation, then selects the best-matching product code from those listed under that regulation. The result is returned in structured JSON format.

2. Detailed Description of Each Cell

Cell 1 — Import Libraries

Imports standard libraries (json and zipfile for reading data, re for extracting ground truth, os/time for file handling and retries) and third-party libraries (numpy for computing cosine similarity, concurrent.futures for concurrent embedding generation, and openai.OpenAI for calling GPT-4.1 and text-embedding-3-small).

Cell 2 — API Connection and Path Configuration

Initializes the OpenAI client (pointing to an Azure deployment endpoint) and defines three global file path variables for direct reference by all subsequent cells.

- **CFR_JSON_PATH:** path to the regulatory database JSON file
- **DATASET_PATH:** path to the compressed archive of the 100-record test dataset
- **EMBED_CACHE:** path to the embedding cache file

The API key field in the saved notebook is left blank and must be filled in manually before running locally.

Cell 3 — Build the CFR Regulatory Index

Traverses the four-level JSON structure (Part → Subpart → Entry → Product Code) to build `cfr_index`, a flat dictionary keyed by product code, used by all subsequent cells.

The text field (used to generate the embedding) is a concatenation of the product title + entry title + the full regulatory text `full_text` (truncated beyond 6,000 characters) + four distinguishing fields. The median length of a regulatory entry is 954 characters, so the vast majority of entries are preserved in full and unaffected by truncation.

Each record in `cfr_index` contains:

- `text` — the concatenated text used to generate the embedding
- `full_text` — the full regulatory text, passed to GPT as the candidate definition
- `devices` — up to 5 historically registered device names, used as reference examples
- `definition` — the product-code-specific definition, used to distinguish sub-codes under the same entry
- `physical_state` — the physical form of the device, used to help distinguish lab/chemical sub-codes
- `technical_method` — the mechanism of action/detection principle, used to help distinguish codes with similar technical methods
- `target_area` — the anatomical or clinical target region, used to help distinguish codes with similar indications

Cell 4 — Define Embedding Utility Functions

Defines two functions used throughout the notebook: `get_embedding(text)` calls `text-embedding-3-small` and returns a 1,536-dimensional vector; `cosine_similarity(vec_a, vec_b)` computes the cosine similarity between two vectors, with a value range of `[-1, 1]`. At the end of this cell, a test call is made using the string “blood pressure monitor” to verify that the API connection works correctly and that the returned vector has the expected length.

Cell 5 — Generate/Load the CFR Vector Cache

Generates embeddings for all 5,955 product codes in `cfr_index` and caches them to `cfr_embeddings.json`. An MD5 hash of the text field is used to check whether the cache is stale (e.g., due to a change in Cell 3 logic); if stale, it is regenerated automatically with no need to manually clear the cache. Generation uses 20 concurrent threads.

Two subsequent unnumbered cells handle gap-filling: they check for and list any product codes whose embeddings failed to generate, then compute embeddings for just those codes and update the cache in place. In this run, the number of missing embeddings was 0.

Cell 6 — Define the Device File Reading Function

Path A (97 records): the `device_description` field has content and is used directly; if `intended_use` also has content, it is appended afterward.

Path B (3 records): `device_description` is empty, so a regular expression extracts a sentence of the form "The [device name] is intended/designed/used for..." from `summary_raw`; if no match is found, it falls back to the Trade/Device Name field.

Ground truth extraction: regardless of which path is used, `true_code` and `true_regulation` are both extracted from `summary_raw` via regular expression and used as the evaluation ground truth. Accuracy is reported separately for the two paths, as well as combined overall.

Cell 7 — Sort the 100 Files

Iterates over all 100 files in the dataset, calling `load_device()` to determine whether each record follows Path A or Path B, and stores them in two separate lists. All 100 files are evaluated; no subset sampling is performed.

Cell 8 — Query Rewriting + Embedding Retrieval

Converts the device description into a vector and retrieves the top 40 candidate product codes with the highest similarity out of 5,955 candidates.

Because commercial-style descriptions differ substantially in style from FDA regulatory language, embedding the raw description directly can cause the correct code to rank low or even fail to appear in the candidate list. To narrow this gap, `rewrite_query()` first uses GPT-4.1 to rewrite the description in a regulatory-language style; the original text and the rewritten version are concatenated and embedded together, capturing both forms of semantic information. Cosine similarity is then computed against every candidate, and the function returns the top 40 along with their respective regulation numbers and titles.

Cell 9 — Install pydantic

Runs `pip install pydantic`, installing the library required to define the structured-output schema in Cell 10. This only needs to be run once per environment.

Cell 10 — Define the GPT Classification Function

Implements the second step of the classification pipeline: the 40 candidates, together with the device description, are sent to GPT-4.1, which makes the decision in two steps — first selecting the regulation, then selecting a product code under that regulation — and returns the result as structured JSON.

Output Format (Pydantic Structured Outputs)

ClassificationResult defines 4 fields, with reasoning placed first to force the model to reason before reaching a conclusion:

Field	Type	Description
reasoning	str	Reasoning process, placed first
selected_regulation_number	str	The regulation selected in Step 1
product_code	str	The product code selected in Step 2; must belong to the regulation selected above
confidence	float	Confidence score (0–1)

Candidate Presentation

The 40 candidates are first grouped by regulation (build_grouped_candidate_text()), then passed to the model. Each candidate code is accompanied by:

- full_text — the full original text of the regulatory entry (the primary basis for judgment)
- definition — the product-code-specific definition, distinguishing neighboring sub-codes under the same entry
- physical_state / technical_method / target_area — supplementary distinguishing fields
- devices (up to 3) — historically registered device names, for reference only

Prompt Strategy

The system prompt requires that: the model must choose from the candidate list and may not return an empty value or a code from a different regulation; it must prioritize the full regulatory definition over titles or examples; the reasoning must first identify the device's characteristics and then compare them against the candidate definitions, explaining why the selected option is better than the runner-up; and even when the description is insufficient, the model must still make a selection, but with confidence below 0.5.

Consistency Check (Key Mechanism)

Structured Outputs only guarantees that the JSON format is valid — it does not guarantee that the selected_regulation_number and product_code fields are mutually consistent (the model may still select code A while reporting regulation B). To address this, after obtaining the result,

the function looks up the actual regulation that `product_code` belongs to using `cfr_index` and compares it against the model's self-reported `selected_regulation_number`; the result is recorded as `regulation_code_consistent`. The final returned `regulation_number` is **always taken from this lookup**, rather than the text freely generated by the model — this way, even if the model's two-step selections are inconsistent, the final output regulation is guaranteed to match the `product_code`.

Cell 11 — Merge the Two Steps into a Single Function

Defines `hybrid_classify()`, which wraps Cell 8 (retrieval) and Cell 10 (classification) into a single unified entry point used by Cell 12 and Cell 13. If the device description is empty, it directly returns a `NO_DESCRIPTION` placeholder result without making any API call.

Cell 12 — Quick Single-Record Test

Before running the full batch evaluation, the complete pipeline is run on the first file in the dataset to confirm that the entire chain — from file reading to returning a prediction — works correctly. The output includes: the regulation selected in Step 1, the product code selected in Step 2, the final regulation number after index-based correction, the confidence score, whether the two-step selections are internally consistent, and the full reasoning text, along with a check of whether the correct answer appears among the top-40 candidates.

Cell 13 — Batch Evaluation (All 100 Files)

Runs `hybrid_classify()` file by file for both Path A and Path B, reporting accuracy separately and overall.

Each record includes:

- `correct` — whether the predicted code matches the true code
- `true_in_top40` — whether the true code appears among the top-40 candidates, used to distinguish retrieval failures from classification errors
- `confidence` — the confidence score
- `reasoning` — the reasoning text
- `regulation_code_consistent` — whether the Step 1 regulation and Step 2 product code are consistent (corresponding to the check logic in Cell 10)

API errors are caught and recorded as `ERROR` without interrupting the evaluation. All results are saved to `results.json`.

Funnel Accuracy

Immediately following Cell 13 is a short, unnumbered cell that reads results.json and computes funnel-style accuracy:

- top-40 retrieval hit rate
- among hits, the proportion with the correct regulation selected
- among correct-regulation cases, the proportion with the correct product code selected
- overall Path A accuracy

Cell 14 — Failure Case Analysis

Aggregates and presents all cases from Cell 13 where the prediction was incorrect, distinguishing two failure modes:

- **Retrieval Failure:** the correct code is not among the top 40 candidates, indicating that the embedding retrieval step failed to recall the correct answer — a retrieval-stage issue.
- **GPT Selection Error:** the correct code is among the candidates, but GPT selected a different code — a classification/prompt-stage issue.

It also tallies how many of the GPT-selection-error cases also had an internal inconsistency between the Step 1 regulation and the Step 2 product code.

Cell 15 — Export the Excel Results Table

Exports all_results to a formatted classification_results.xlsx, containing all 14 fields (K Number, Path, device description, true/predicted code and regulation, consistency check result, confidence, reasoning text, etc.), with the header row frozen and word wrap enabled.

Each row is color-coded by result type:

Color	Meaning
☑ Green	Correct prediction
☒ Orange	GPT selection error (true code among the top-40 candidates, but the model selected a different one)
☒ Red	Retrieval failure (true code not among the top-40 candidates)
Gray	API call error, skipped (did not occur in this 100-record evaluation)

3. Output Files

This notebook is fairly long and involves many cells; the corresponding output files are summarized together below. Of these, results.json is the core output, storing the predictions for all 100 records. Both the subsequent funnel accuracy calculation (the unnumbered cell after Cell 13) and the failure case analysis (Cell 14) read this file directly and reuse it, rather than re-

running the classification pipeline.

File	Produced By	Contents
cfr_embeddings.json	Cell 5	The embedding vector cache for each of the 5,955 product codes, plus the hash fingerprint used to determine whether the cache needs to be regenerated
results.json	Cell 13	One record per device (100 total), including the predicted code, predicted regulation, ground truth, confidence, reasoning text, and the retrieval/consistency-related flag fields described above
classification_results.xlsx	Cell 15	A formatted, color-coded version of results.json for manual review; rows are highlighted by result type (correct, retrieval failure, GPT selection error, API skipped)

3. Results

All figures in this section are computed directly from results.json and have been cross-checked against classification_results.xlsx.

1. Overall Accuracy

	Correct	Total	Accuracy
Overall	64	100	64.0%
Path A (has device description)	61	97	62.9%
Path B (description extracted from summary)	3	3	100.0%

Path B accounts for only 3 of the 100 records, so its 100% figure is not treated as a meaningful comparison against Path A.

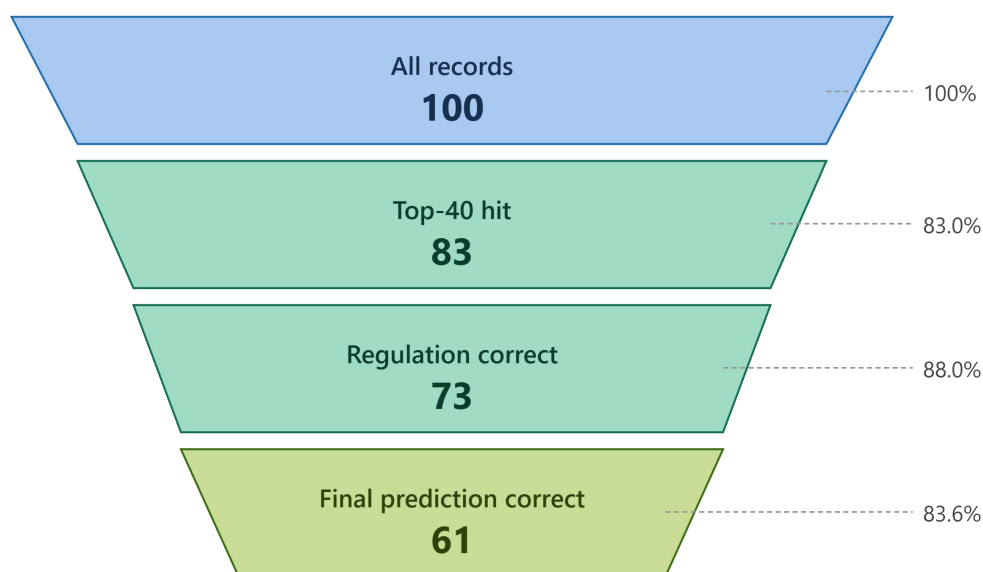
2. Funnel Breakdown

The overall pipeline makes predictions in two stages — retrieval, then classification — and the classification stage itself has two steps: first selecting the regulation, then selecting the product code under it. Breaking accuracy down by stage shows exactly where the 36 errors occurred.

- **Stage 1 — Top-40 retrieval:** of the 100 records, 83 (83.0%) had the correct product code appear among the top-40 candidates returned by embedding retrieval.

- **Stage 2 — Regulation selection:** among these 83 hit cases, the model selected the correct regulation in 73 (88.0%).
- **Stage 3 — Code selection:** among these 73 correct-regulation cases, the model selected the correct product code in 61 (83.6%).

This is shown visually in the figure below.



3. Internal Consistency Check

The classification step requires the model to first select a regulation, then select a product code that belongs to that regulation. Because Structured Outputs only guarantees that the returned JSON is syntactically valid — not that these two fields are mutually consistent — every prediction underwent an internal consistency check: a lookup against the regulatory database confirms whether the returned product code actually belongs to the returned regulation number. Across all 100 records, the regulation selected in Step 1 and the product code selected in Step 2 were mutually consistent in every case (regulation_code_consistent was true in 100/100 records).

4. Failure Attribution

Of the 100 predictions, 36 were incorrect. Based on whether the correct answer ever had a chance of being selected, each error falls into one of two categories:

- **Retrieval failure (17 cases):** the correct product code never appeared among the top-40 candidates, so the classification step never had a chance to select it.
- **GPT selection error (19 cases):** the correct product code appeared among the top-40 candidates, but the model selected a different code.

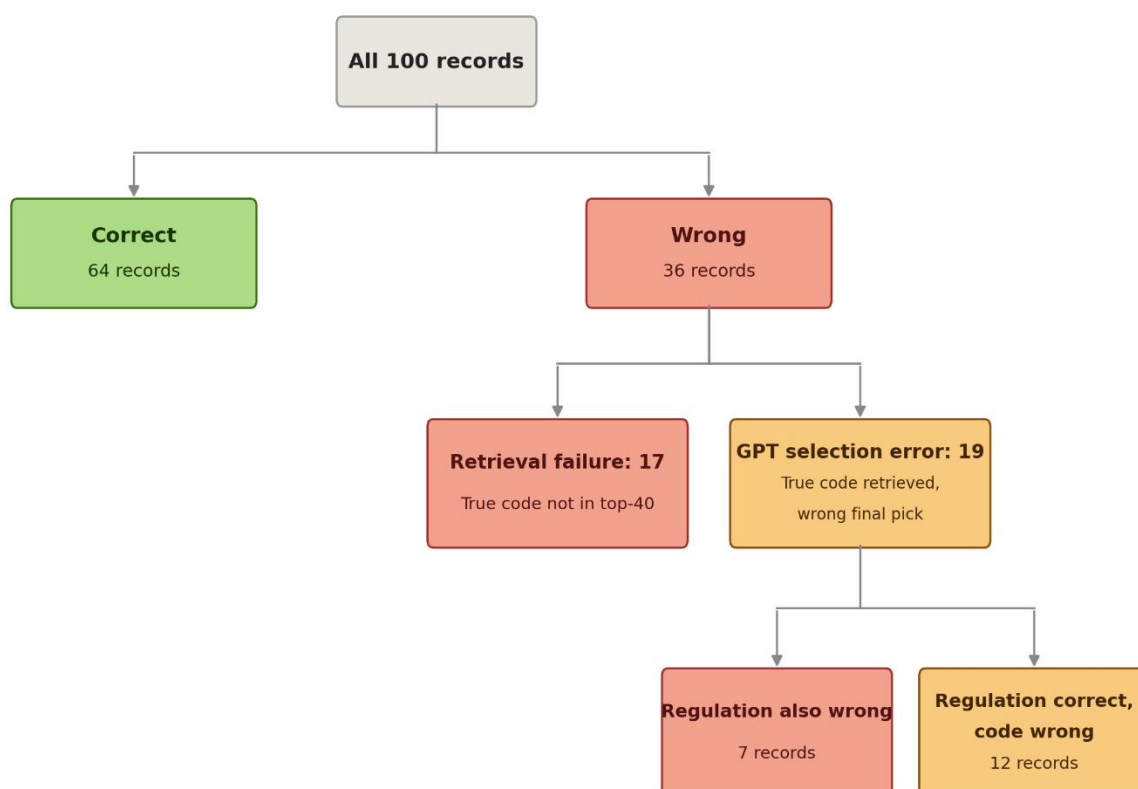
These 19 GPT selection errors can be further broken down by whether the regulation itself was selected correctly:

- **12 cases** — the regulation was selected correctly, but the wrong specific product code was chosen from those listed under it.
- **7 cases** — the regulation itself was selected incorrectly.

This 12/7 split is consistent with the fact that many regulation entries in the regulatory database have multiple product codes attached: once the model successfully narrows the scope to the correct regulation, the remaining difficulty (12 of the 19 GPT errors) is concentrated mainly in distinguishing between neighboring codes under the same regulation.

The two major failure categories (17 vs. 19) are fairly close in number, suggesting that in this run, retrieval and classification contributed roughly equally to the total error count, with neither stage clearly dominating.

This is shown visually in the figure below.



5. Confidence Calibration

	Average Confidence	Sample Size (n)
Correct predictions	0.93	64
Incorrect predictions	0.84	36

Confidence does not reliably distinguish correct from incorrect predictions. Of the 36 incorrect predictions, 31 (86%) still had a confidence of 0.8 or above — the same range occupied by nearly all correct predictions. In this run, no correct prediction had a confidence below 0.5, so low confidence can serve as a meaningful signal that a prediction may be wrong, but high confidence cannot serve as a meaningful signal that a prediction is correct.

6. Representative Cases

Retrieval failure case — K252528

The Ion™ Endoluminal System (model IF1000), a software-controlled electromechanical system. The correct code, EOQ (21 CFR 874.4680), never appeared among the top-40 candidates; the model predicted QZB (876.1500), from a completely unrelated regulation, with a confidence of 0.9.

GPT selection error, regulation correct — K252526

Rapid DeltaFuse, a Software as a Medical Device (SaMD) imaging-processing module. In Step 1, the model correctly selected regulation 21 CFR 892.2050 (matching the correct answer), but in Step 2 it selected product code QIH instead of the correct code, LLZ — these two codes happen to fall under the same regulation. The confidence was 0.78, the lowest of the three cases shown in this section.

GPT selection error, regulation also incorrect — K252443

SpaceFix Shoulder Spacer, a temporary shoulder spacer made of gentamicin-containing PMMA bone cement. The model selected regulation 888.3027 and code MBB, while the correct answer was regulation 21 CFR 888.3660 and code KWS — a completely different regulation. Even though both fields were predicted incorrectly, the confidence was still 0.9, consistent with the confidence calibration findings in Section 5.

4. Observations

The findings below are all drawn from results.json, classification_results.xlsx, and the notebook source code.

1. Errors are concentrated in a small number of regulations rather than randomly distributed.

Of the 36 errors, 18 are concentrated across just 8 regulations (each contributing 2–3 errors). Two recurring confusion patterns stand out in particular: under 21 CFR 892.2050, the correct code LLZ was twice mispredicted as QIH (K252526, K252608); under 21 CFR 874.4680, the correct code EOQ was twice mispredicted as QZB (K252528, K252045). 21 CFR 878.4400 also produced 3 errors, all with correct code GEI but mispredicted as three different codes. This indicates that some errors are systematic, reproducible confusion patterns rather than independent random noise.

2. Accuracy varies considerably across CFR Parts, but sample sizes are too small to draw conclusions.

Grouped by Part, accuracy ranges from 0% (Part 868, $n=3$) to 86% (Parts 880/882, both 6/7), but most Parts have fewer than 15 samples, and some have only 1–4. With such small sample sizes, one or two errors can swing accuracy by more than 30 percentage points, so Part-level differences more likely reflect “this particular batch happened to include difficult cases” rather than that Part itself being inherently harder.

3. Confidence is generally high and shows no reliable relationship with correctness or reasoning length.

Among the 36 errors, 86% still had confidence ≥ 0.8 . Reasoning text length shows a weak negative correlation with confidence ($r = -0.21$), and the average reasoning length for incorrect predictions (395.6 words) was slightly longer than for correct predictions (360.2 words) — consistent with the intuition that “difficult cases require more comparative text,” but conversely confirming that neither high confidence nor long reasoning can serve as a proxy for correctness.

4. The two-step design performed as expected in this run.

The regulation/code internal consistency check passed for all 100 predictions (zero contradictions). Of the 19 GPT selection errors, 12 (63%) involved a correctly selected regulation with only the code chosen incorrectly — precisely the failure mode the two-step design targets, since 41.6% of regulations in the CFR database (875 of 2,101) have more than one product code attached. The remaining 7 cases involved an incorrectly selected regulation — a harder failure mode not directly addressed by this design.

5. Future Directions

1. Retrieval Stage (addressing the 17 retrieval failures)

- **Combine lexical matching signals (e.g., BM25) with dense embeddings (e.g., via reciprocal rank fusion).** In some of this run's retrieval failure cases, the device naming (e.g., "Ion™ Endoluminal System") had almost no overlap with CFR regulatory wording, making it difficult for dense embeddings to capture exact terminology matches; lexical retrieval could serve as a complementary signal.
- **Add a reranking stage between retrieval and classification.** Using a cross-encoder to re-score and re-rank the top-40 candidates costs far less than scoring all 5,955 candidates, and could push the correct regulation more reliably toward the top of the list.

2. Classification Stage (addressing the 12 cases where the regulation was correct but the code was wrong)

- **Split the two-step decision into two separate API calls**, first selecting the regulation, then selecting the code only from among those under that regulation, to avoid the model's attention being diluted by the other 30-plus irrelevant candidates when making the finer-grained judgment.
- **Explore self-consistency (multiple sampling + majority voting) for the code selection step.** This sub-task's accuracy (83.6%) is lower than that of regulation selection (88.0%), and confidence has already been shown to be unreliable; sampling-based voting offers an alternative that does not depend on the model's self-reported confidence.